UNIVERSITÀ DEGLI STUDI
DI SALERNO

Laurea Magistrale in Ingegneria Informatica

Biometry and Behavior Analysis 2025/2026 - Gruppo 11

Face Recognition System

Report del Progetto

Cipriano Ivan Luigi	0622702507
Faraulo Simone	0622702509

Contents

1 Panoramica e Architettura del Sistema **2**
1.1 Pipeline di Elaborazione . . . 2
1.2 Scelte Progettuali e Contributi Principali . . . 2

2 Modulo di Acquisizione e Pre-processing **2**
2.1 Rilevamento del Volto e Allineamento . . . 3
2.2 Stima Geometrica della Posa (Yaw e Pitch) . . . 3
2.2.1 Controllo Rotazione Orizzontale (Yaw) . . . 3
2.2.2 Controllo Rotazione Verticale (Pitch) . . . 3
2.3 Metriche di Qualità dell'Immagine . . . 3
2.4 Vincolo Dimensionale per la Robustezza della Liveness . . . 4
2.5 Strategia di Acquisizione "Best-Valid" . . . 4

3 Rilevamento Liveness e Anti-Spoofing **4**
3.1 Composizione e Gestione del Dataset . . . 4
3.1.1 Strategia di Split Anti-Leakage: Subject & Video Disjoint . . . 5
3.2 Architettura del Modello e Training . . . 5
3.2.1 Bilanciamento delle Classi e Augmentation . . . 5

4 Feature Extraction e Matching **5**
4.1 Architettura del Modello: InceptionResnetV1 . . . 5
4.2 Gestione della Gallery e Privacy . . . 6
4.3 Logica di Matching: Distanza Euclidea . . . 6
4.4 Processo di Enrollment . . . 7

5 Calcolo delle Soglie e Valutazione Sperimentale **7**
5.1 Modulo Liveness: Selezione della Soglia T_f . . . 7
5.1.1 Risultati Liveness sul Test Set . . . 8
5.1.2 Valutazione della Robustezza agli Attacchi (System-Level) . . . 8
5.2 Modulo di Verifica (1:1): Selezione della Soglia T_v . . . 8
5.2.1 Metodologia di Calcolo dei Punteggi . . . 8
5.2.2 Risultati Verifica sul Test Set . . . 9
5.3 Modulo di Identificazione (1:N): Selezione della Soglia T_i . . . 9
5.3.1 Metodologia di Calcolo degli Score . . . 10
5.3.2 Risultati Identificazione sul Test Set . . . 10

6 Requisiti di Sistema e Conclusioni **11**
6.1 Analisi delle Performance Computazionali . . . 11
6.2 Esperimenti Fallimentari e Lezioni Apprese . . . 11

1 Panoramica e Architettura del Sistema

Il sistema gestisce tre task primari: Enrollment, Identificazione Open-set e Verifica, integrati con un modulo di Replay Attack Detection. La pipeline è progettata per scartare campioni di bassa qualità nelle prime fasi del processo e ottimizzare il trade-off tra False Acceptance Rate (FAR) e False Rejection Rate (FRR) tramite soglie derivate sperimentalmente.

1.1 Pipeline di Elaborazione

Il flusso dei dati è strutturato in quattro stadi sequenziali logici. Ogni stadio agisce come un filtro: solo i dati che soddisfano le post-condizioni passano allo stadio successivo.

Table 1: Definizione dei moduli della pipeline: Input, Logica di Elaborazione e Post-condizioni.

Modulo / Fase	Input e Pre-condizioni	Logica e Post-condizioni (Output)
1. Acquisizione	Stream video RGB. *Pre-cond:* Presenza di un volto rilevabile da MTCNN.	*Post-cond:* Face Crop 160×160 frontale ($\pm 15°$) e nitido. Se fallisce: scarto e nuovo tentativo.
2. Liveness Detection	Face Crop validato. *Pre-cond:* Modulo attivato da configurazione.	*Post-cond:* Score di liveness $S > T_f$. Se $S \leq T_f$: Reject immediato (Replay Attack).
3. Feature Extraction	Face Crop "Live". *Pre-cond:* Nessuna (processo deterministico).	*Post-cond:* Vettore di embedding $x \in R^{512}$ normalizzato ($\|\|x\|\|_2 = 1$).
4. Matching	Embedding Probe (p) e Gallery (G). *Pre-cond:* Gallery non vuota.	*Post-cond:* ID utente, Match (Bool) o Reject status.

1.2 Scelte Progettuali e Contributi Principali

Per garantire robustezza e riproducibilità, sono state implementate specifiche scelte progettuali:

- **Stima Geometrica della Posa 3D da Landmark 2D:** A differenza del filtraggio standard basato su bounding-box, abbiamo implementato un'euristica geometrica che stima gli angoli di Yaw e Pitch utilizzando i rapporti delle distanze tra i landmark facciali (occhi-naso-bocca). Questo assicura che vengano processati solo volti frontali, riducendo la varianza intra-classe.
- **Training Liveness Stratificato:** Il modulo anti-spoofing utilizza una ResNet50 addestrata con strategia "Partial Fine-Tuning" (congelamento backbone, training layer 2-3-4 + FC), utilizzando dataset privato e un dataset pubblico. Per evitare data leakage, lo split del dataset è stato stratificato basandosi sull'ID del video sorgente, garantendo che frame dello stesso video non appaiano contemporaneamente in train e test.
- **Ottimizzazione Spazio delle Feature:** La gallery memorizza esclusivamente gli embedding normalizzati. Le soglie T_i e T_v sono state calibrate sul Validation Set.

2 Modulo di Acquisizione e Pre-processing

Il modulo di acquisizione (`acquisition.py`) rappresenta il primo filtro critico del sistema. L'obiettivo non è semplicemente catturare un volto, ma garantire che il campione in ingresso possieda requisiti geometrici e

fotometrici sufficienti per massimizzare l'accuratezza del riconoscimento facciale a valle. Abbiamo implementato una strategia "Best-of-N" con scarto attivo dei campioni non cooperativi.

2.1 Rilevamento del Volto e Allineamento

Per la fase di detection, il sistema utilizza la rete **MTCNN**. A differenza dei rilevatori statici, MTCNN offre una maggiore invarianza alle condizioni di illuminazione e scala, fornendo inoltre 5 landmark facciali chiave (occhio dx/sx, naso, bocca dx/sx) essenziali per la nostra stima della posa. Il sistema impone un vincolo rigoroso *Single-Face*: se vengono rilevati più volti ($N_{faces} > 1$), l'acquisizione viene invalidata per evitare ambiguità sull'identità del soggetto.

2.2 Stima Geometrica della Posa (Yaw e Pitch)

Per garantire la robustezza del matching senza l'overhead computazionale di una rete neurale dedicata alla stima della posa, abbiamo implementato un algoritmo geometrico leggero basato sui rapporti tra i landmark 2D forniti da MTCNN.

Siano $P_{le}, P_{re}, P_n, P_{lm}, P_{rm}$ le coordinate (x, y) rispettivamente di occhio sx, occhio dx, naso, bocca sx e bocca dx.

2.2.1 Controllo Rotazione Orizzontale (Yaw)

La simmetria orizzontale viene valutata confrontando le distanze orizzontali del naso rispetto agli angoli della bocca. Definiamo lo score di asimmetria S_{yaw}:

$$d_{lx} = |P_n.x - P_{lm}.x|, \quad d_{rx} = |P_{rm}.x - P_n.x| \tag{1}$$

$$S_{yaw} = \frac{|d_{lx} - d_{rx}|}{d_{lx} + d_{rx}} \tag{2}$$

Un valore di $S_{yaw} \approx 0$ indica un volto perfettamente frontale. Il sistema scarta i campioni con $S_{yaw} > 0.30$, che corrisponde a una rotazione stimata superiore a $\approx 15° - 20°$.

2.2.2 Controllo Rotazione Verticale (Pitch)

La rotazione verticale viene stimata analizzando il rapporto tra la parte superiore e inferiore del volto. Calcoliamo i punti medi degli occhi M_{eyes} e della bocca M_{mouth}, e le relative distanze euclidee dal naso:

$$R_{pitch} = \frac{||M_{eyes} - P_n||_2}{||P_n - M_{mouth}||_2} \tag{3}$$

Sperimentalmente, abbiamo determinato che un volto frontale mantiene $R_{pitch} \approx 1.0$.

- Se $R_{pitch} < 0.35$: Il soggetto guarda in **ALTO** (il naso si avvicina prospetticamente agli occhi).
- Se $R_{pitch} > 1.80$: Il soggetto guarda in **BASSO**.

Il range di accettazione è definito come $R_{pitch} \in [0.35, 1.80]$.

2.3 Metriche di Qualità dell'Immagine

Oltre alla posa, ogni crop facciale viene sottoposto a un controllo di qualità fotometrica per evitare che immagini sfocate o mal esposte degradino il vettore di feature. Le metriche calcolate sono riportate in Tabella 2.

Table 2: Soglie di qualità implementate in `acquisition.py`. I valori sono stati tarati empiricamente, tutti i test sono presenti nel file `acquisition_test.ipynb`.

Metrica	Algoritmo	Soglia / Range	Motivazione
Blur	Varianza del Laplaciano (σ^2_Δ)	> 100.0	Scarta motion blur elevato.
Contrasto	Deviazione Standard (σ_{gray})	> 40.0	Evita immagini "piatte".
Luminosità	Intensità Media (μ_{gray})	$[40, 220]$	Evita sotto/sovraesposizione.
Min Face Size	Lato minore Bounding Box	> 450 px	Risoluzione minima per CNN.

2.4 Vincolo Dimensionale per la Robustezza della Liveness

Un'importante deviazione rispetto ai parametri standard di MTCNN è l'impostazione di `min_face_size = 450` pixel. Questa scelta progettuale deriva da un'analisi sperimentale degli errori del modulo di Liveness Detection. Durante la fase di validazione, abbiamo osservato un alto tasso di falsi negativi (soggetti *Live* classificati come *Spoof*) quando il soggetto si trovava distante dalla fotocamera. Il classificatore di Liveness, addestrato su attacchi di tipo Replay (foto di foto o schermi), tende a correlare la bassa risoluzione e la perdita di dettagli ad alta frequenza con la classe *Spoof*. Imponendo una dimensione minima elevata (450×450 px), il sistema forza l'utente ad avvicinarsi al sensore durante l'acquisizione. Questo accorgimento normalizza la risoluzione spaziale dell'input, garantendo che il classificatore basi la decisione esclusivamente sulla presenza di artefatti di texture genuini e non sulla degradazione dell'immagine dovuta al downscaling o alla distanza.

2.5 Strategia di Acquisizione "Best-Valid"

Il processo di acquisizione è iterativo (max 3 tentativi). Per ogni frame:

1. Viene estratto il volto e calcolate le metriche geometriche e fotometriche.
2. Se il volto supera **tutti** i controlli, l'acquisizione termina immediatamente con successo.
3. Se nessun tentativo è perfetto, il sistema implementa una strategia di *fallback*: viene restituito il campione con il miglior punteggio combinato $Score = Blur \times Contrast$, a patto che sia rilevabile, minimizzando il rischio di mancata acquisizione pur segnalando l'anomalia.

3 Rilevamento Liveness e Anti-Spoofing

E' stato sviluppato un modulo di classificazione binaria che permette di classificare un volto come Live o Spoof. Questo modulo se attivo permette di far passare solo i volti classificati come "Live".

3.1 Composizione e Gestione del Dataset

Per l'addestramento è stato curato un dataset ibrido composto da due sorgenti:

- **Dataset Proprietario:** Abbiamo utilizzato le foto del nostro dataset di 20 soggetti. Per generare campioni di classe *Spoof*, abbiamo riprodotto le foto dei soggetti su monitor (PC e Smartphone), ri-acquisendoli con tramite fotocamera per simulare un attacco reale.
- **Dataset Esterno (Kaggle):** Per aumentare la capacità di generalizzazione e prevenire l'overfitting sui soli 20 soggetti, abbiamo integrato il dataset proprietario con campioni provenienti dall'*Anti-Spoofing Dataset Maker*, considerando solamente frame acquisiti da video che inquadrano soggetti reali e video di replay attack effettuando un filtraggio tramite le label del csv.

3.1.1 Strategia di Split Anti-Leakage: Subject & Video Disjoint

Per mitigare il problema del *Data Leakage*, per cui frame dello stesso soggetto finiscono sia in Training e Test set, abbiamo adottato due strategie complementari:

- **Subject-Disjoint Split (Dataset Proprietario):** per il dataset interno, abbiamo imposto un vincolo rigoroso sull'identità. Tutti i campioni relativi a un soggetto (comprendenti **sia le Gallery che le Probe**, e tutti i relativi attacchi Live/Spoof) sono stati assegnati in blocco ad un unico split. Nello specifico, la ripartizione dei 20 soggetti è stata: **11 nel Training Set, 2 nel Validation Set e 7 nel Test Set**. Questo garantisce una valutazione realistica su identità mai viste (*unseen subjects*).
- **Video-Based Stratified Split (Dataset Esterno):** Per i dati provenienti da Kaggle, lo split (Train 70%, Val 20%, Test 10%) è stato effettuato sulla lista univoca dei file video tramite `liveness_detector.py`. Questo assicura che frame estratti dallo stesso video non siano condivisi tra gli split, prevenendo l'overfitting temporale.

3.2 Architettura del Modello e Training

Come rete è stata utilizzata una **ResNet50** pre-addestrata su ImageNet. Data la natura specifica del task, abbiamo adottato una strategia di **Partial Fine-Tuning**:

- **Backbone Congelata**.
- **Layer Addestrabili:** Solo gli ultimi tre blocchi convoluzionali (`layer2`, `layer3`, `layer4`) e il livello Fully Connected (FC) finale sono stati resi addestrabili. Questo permette alla rete di adattare le feature semantiche di alto livello al dominio dello spoofing senza distruggere la conoscenza pregressa.

3.2.1 Bilanciamento delle Classi e Augmentation

Dall'analisi esplorativa è emerso uno sbilanciamento tra le classi (numero di video *Spoof* > *Real*). Abbiamo mitigato il problema tramite:

1. **Frame Extraction Bilanciata:** Durante la creazione del dataset, il numero di frame estratti dai video *Live* è stato moltiplicato per un fattore $K = N_{spoof}/N_{live}$ per pareggiare il numero di campioni.
2. **Data Augmentation:** Sul training set sono state applicate trasformazioni randomiche aggressive: *Horizontal Flip*, *Rotation* ($\pm 15°$) e *ColorJitter* (luminosità/contrasto) per rendere il modello robusto alle variazioni di illuminazione ambientale.

Il training è stato gestito tramite le classi custom `Trainer` e `CheckpointManager`, utilizzando l'ottimizzatore **Adam** ($lr = 10^{-4}$) e la funzione di costo **BCEWithLogitsLoss**. Per prevenire l'overfitting, è stato utilizzato uno scheduler *ReduceLROnPlateau* che riduce il learning rate se la loss di validazione non migliora per 5 epoche consecutive.

4 Feature Extraction e Matching

Il modulo di estrazione delle feature è responsabile della trasformazione dell'immagine del volto in una rappresentazione vettoriale compatta (embedding) che sia invariante a piccole variazioni di posa e illuminazione, ma altamente discriminativa tra identità diverse.

4.1 Architettura del Modello: InceptionResnetV1

Per l'estrazione delle feature abbiamo adottato l'architettura **InceptionResnetV1**, pre-addestrata sul dataset VGGFace2. La scelta di questo modello rappresenta un ottimo compromesso tra accuratezza e costo computazionale, essenziale per rispettare il vincolo di memoria (< 4GB RAM).

L'implementazione è definita nella classe `FaceRecognitionSystem` in `feature_extractor.py`. Il processo di inferenza avviene nei seguenti step:

1. **Preprocessing:** L'immagine croppata (160×160 px) viene convertita in tensore e normalizzata con media $[0.5, 0.5, 0.5]$ e deviazione standard $[0.5, 0.5, 0.5]$, portando i valori dei pixel nel range $[-1, 1]$.
2. **Embedding:** La rete proietta l'immagine in uno spazio euclideo a 512 dimensioni.
3. **Normalizzazione L_2:** L'output grezzo della rete viene normalizzato:

$$f(x) = \frac{v(x)}{||v(x)||_2} \quad \text{dove } v(x) \in R^{512} \tag{4}$$

   Questo passaggio, visibile nel codice `feature_extractor.py` tramite `torch.nn.functional.normalize`, è fondamentale perché permette di utilizzare la Distanza Euclidea come metrica diretta di similarità semantica, in quanto è equivalente alla similarità coseno.

4.2 Gestione della Gallery e Privacy

Una scelta progettuale critica riguarda la memorizzazione dei dati biometrici. Per motivi di **Privacy** e efficienza di memoria, il sistema **NON memorizza le immagini originali** dei volti nella Gallery. Durante la fase di *Enrollment*, le feature vengono estratte e salvate in file binari `.npy`. La Gallery è costituita esclusivamente da due array:

- `gallery_embeddings`: matrice di dimensione $((N_{samples} \cdot N_{subjects}) \times 512)$.
- `gallery_labels`: vettore di dimensione $(N_{samples})$ contenente gli ID numerici.

Questa struttura permette di caricare l'intera gallery in VRAM (GPU) o RAM all'avvio, consentendo operazioni di matching vettorializzate estremamente rapide tramite `torch.cdist`.

4.3 Logica di Matching: Distanza Euclidea

La scelta della metrica di confronto è stata dettata dal requisito funzionale di **Online Enrollment**. Poiché il sistema deve permettere la registrazione di nuovi utenti direttamente in fase di inferenza, l'utilizzo di un classificatore neurale è stato scartato, in quanto avrebbe richiesto il riaddestramento della rete ad ogni nuovo inserimento. Tale operazione violerebbe il vincolo di eseguire l'intera pipeline in meno di 60 secondi.

Abbiamo pertanto optato per un approccio dove il confronto tra il volto acquisito (Probe, p) e i volti in memoria (Gallery, G) avviene calcolando direttamente la distanza Euclidea a coppie nello spazio delle feature:

$$d(p, g_i) = ||p - g_i||_2 \tag{5}$$

Il sistema, implementato in `biometric_system.py`, gestisce due modalità operative distinte basate su soglie sperimentali (T_i e T_v):

- **Open-set Identification (1:N):** Il sistema calcola la distanza tra p e tutti i vettori in G.
  - Viene identificato l'indice k tale che $d(p, g_k)$ sia minimo.
  - Se $\min(d) < T_i$ (Soglia Identificazione), l'utente è riconosciuto come l'ID associato a g_k.
  - Se $\min(d) \geq T_i$, il sistema restituisce **REJECT** (utente sconosciuto).
- **Verification (1:1):** L'utente dichiara un'identità (ID_{claim}). Il sistema filtra la Gallery considerando solo i sotto-vettori $G_{claim} \subset G$ associati a quell'ID.
  - Viene calcolata la distanza minima solo rispetto a G_{claim}.

- Se $\min(d) < T_v$ (Soglia Verifica), l'accesso è garantito. Altrimenti, è negato.

L'utilizzo di `torch.cdist` permette di parallelizzare questi calcoli sulla GPU, mantenendo il tempo di inferenza post-acquisizione ampiamente sotto i limiti temporali richiesti.

4.4 Processo di Enrollment

L'aggiunta di un nuovo utente al sistema avviene in tempo reale (*Online Enrollment*). Il processo segue questi passaggi: **Acquisizione:** L'utente viene sottoposto alla pipeline di acquisizione per catturare un volto frontale di alta qualità.

Feature Extraction: Dal volto validato viene estratto il vettore di embedding $v_{new} \in R^{512}$ tramite Inception-ResnetV1.

Aggiornamento Gallery: Il sistema assegna un nuovo ID univoco (ID_{new}) e aggiunge la coppia (v_{new}, ID_{new}) ai file .npy. Questo approccio permette di estendere il database senza dover riaddestrare alcun modello neurale, garantendo scalabilità immediata.

5 Calcolo delle Soglie e Valutazione Sperimentale

La calibrazione del sistema biometrico si basa sulla determinazione di tre soglie critiche: T_i (Identification), T_v (Verification) e T_f (Liveness). Per garantire una valutazione imparziale e prevenire l'*overfitting*, abbiamo adottato una strategia di **Hold-Out Validation** differenziata per modulo:

- **Per la Liveness:** Abbiamo utilizzato un Validation Set *ibrido*, composto sia da immagini del nostro dataset proprietario (Live e Spoof) sia da campioni del dataset pubblico. Questo approccio garantisce che la soglia T_f sia robusta e generalizzi bene.
- **Per Verifica e Identificazione:** Abbiamo utilizzato *esclusivamente* le immagini di Probe del nostro dataset proprietario (20 soggetti). Il set di Probes è stato partizionato (40% Validation, 60% Test) per calcolare le soglie T_i e T_v.

5.1 Modulo Liveness: Selezione della Soglia T_f

L'obiettivo primario del modulo anti-spoofing è la sicurezza. In `spoofing_evaluator.py`, sfruttando il Validation Set ibrido, abbiamo calcolato due candidate soglie:

1. **Soglia EER ($T_{eer} = 0.5962$):** Punto di equilibrio dove $FAR \approx FRR$.
2. **Soglia Security-First ($T_{sec} = 0.6459$):** Calcolata fissando un target $FAR \leq 0.5\%$, per minimizzare drasticamente il rischio di accessi fraudolenti.

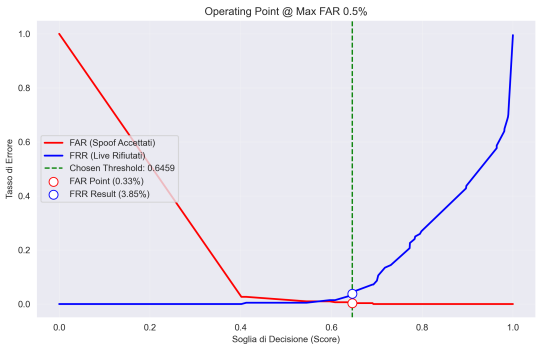

Figure 1: Soglia di liveness sul validation set.

Decisione: La selezione della soglia non è avvenuta cercando il generico equilibrio (EER), che considera un falso accesso grave quanto un falso rifiuto. Abbiamo invece modellato il problema con un **costo dell'errore asimmetrico**. Sul Validation Set, la soglia EER ($T_{eer} = 0.5962$) garantiva un FAR dell'1.00%. Spostando la soglia $T_f = 0.6459$ (Target $FAR \leq 0.5\%$), abbiamo ridotto la probabilità di successo degli attacchi simulati allo **0.33%**. Questa scelta progettuale accetta consapevolmente un degrado dell'usabilità (FRR sul validation salito dall'1.44% al 3.85%), che viene mitigato dalla natura interattiva del sistema: un utente legittimo rifiutato può ripetere l'acquisizione in pochi secondi (come previsto dai requisiti), mentre un attaccante bloccato rappresenta una minaccia neutralizzata.

5.1.1 Risultati Liveness sul Test Set

La valutazione finale, condotta sul Test Set (364 campioni bilanciati), conferma la validità della scelta effettuata sul set ibrido.

Table 3: Performance Liveness (Test Set, $T_f = 0.6459$).

Metrica	Valore	Note Operative
Accuracy	90.93%	-
FAR (Spoof Accettati)	9.34%	17 attacchi riusciti su 182.
FRR (Live Rifiutati)	8.79%	16 utenti bloccati su 182.
HTER	9.07%	Media equilibrata degli errori.

Analisi degli Errori: Analizzando i 16 casi di *False Rejection* (utenti reali bloccati), abbiamo notato che la maggioranza riguarda campioni con forti riflessi sugli occhiali o condizioni di controluce estremo, che la rete confonde con la lucentezza tipica degli schermi LCD. Al contrario, i 17 *False Acceptances* (attacchi riusciti) provengono quasi esclusivamente da foto ad alta risoluzione.

5.1.2 Valutazione della Robustezza agli Attacchi (System-Level)

Per quantificare il reale impatto del modulo di sicurezza, abbiamo misurato il **System Rank-1 FAR** sotto attacco Replay, confrontando il sistema con e senza protezione attiva, utilizzando il nostro dataset di test privato. **Scenario A (Liveness OFF):** Senza filtro, il sistema di riconoscimento facciale è estremamente vulnerabile. Su 182 attacchi video, ben 181 sono stati erroneamente accettati come utenti legittimi, portando il **Rank-1 FAR al 99.45%**. Questo dimostra che il solo matching biometrico non distingue un volto reale da uno schermo. **Scenario B (Liveness ON):** Attivando il modulo con soglia $T_f = 0.6459$, il sistema ha bloccato 165 attacchi su 182. I 17 attacchi residui che hanno superato il filtro sono stati poi processati dal matcher. Il **System Rank-1 FAR scende al 9.34%**, riducendo il successo di attacco di oltre un ordine di grandezza.

5.2 Modulo di Verifica (1:1): Selezione della Soglia T_v

Il task di Verifica consiste nel confermare se un utente è chi dice di essere. Per determinare la soglia operativa T_v, abbiamo adottato un protocollo rigoroso basato sulla partizione del set di Probes, mantenendo la Gallery come riferimento fisso (Ground Truth). Abbiamo utilizzato il **40% delle Probes** (Validation Set) per modellare le distribuzioni di probabilità delle distanze, lasciando il restante 60% (Test Set) intatto per la valutazione finale.

5.2.1 Metodologia di Calcolo dei Punteggi

Sfruttando il Validation Set, abbiamo generato due liste di punteggi (distanze) distinte per simulare tutti i possibili scenari operativi:

- **Genuine Scores (Intra-classe):** Abbiamo calcolato la distanza euclidea minima tra ogni Probe del Validation Set e i template in Gallery appartenenti allo *stesso* soggetto ($Probe_{ID=X} \leftrightarrow Gallery_{ID=X}$). Questi valori rappresentano la variabilità naturale del volto di un utente legittimo.

- **Impostor Scores (Inter-classe):** Abbiamo confrontato ogni Probe del Validation Set con i template in Gallery di *tutti gli altri* soggetti ($Probe_{ID=X} \leftrightarrow Gallery_{ID \neq X}$). Questi valori rappresentano i tentativi di accesso fraudolento.

Analizzando queste due distribuzioni, abbiamo calcolato i tassi di errore al variare della soglia t:

- **FAR (False Acceptance Rate):** La frazione di *Impostor Scores* inferiori a t.
- **FRR (False Rejection Rate):** La frazione di *Genuine Scores* superiori a t.

La soglia ottimale è stata identificata nel punto di **Equal Error Rate (EER)**, dove le curve FAR e FRR si intersecano ($FAR \approx FRR$). Sul Validation Set, questo punto è stato trovato a $T_v = 0.7909$, con un EER teorico dello 0.70%.

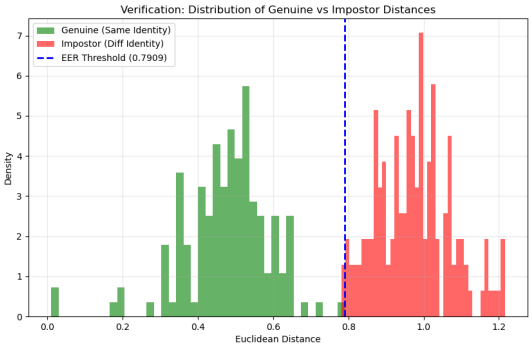

Figure 2: Distribuzione delle distanze minime verification sul validation set.

5.2.2 Risultati Verifica sul Test Set

Applicando la soglia $T_v = 0.7909$ al **Test Set indipendente** (il restante 60% delle Probes, corrispondente a 216 campioni mai utilizzati per la calibrazione), il sistema ha ottenuto i seguenti risultati:

- **FAR (False Acceptance Rate): 0.00%**. Nessun tentativo di impostore (incrocio Probe X vs Gallery Y) ha prodotto una distanza inferiore alla soglia.
- **FRR (False Rejection Rate): 0.46%**. Solo lo 0.46% dei tentativi legittimi è stato erroneamente rifiutato.
- **Rank-1 Accuracy: 99.54%**.

Questi risultati eccellenti, superiori all'EER stimato in validazione, confermano che le feature estratte sono altamente discriminanti e che il rigido controllo sulla posa in fase di acquisizione ha ridotto drasticamente la varianza intra-classe.

5.3 Modulo di Identificazione (1:N): Selezione della Soglia T_i

Nello scenario di Identificazione Open-Set, il sistema deve affrontare una doppia sfida: classificare correttamente gli utenti registrati (*Known Users*) e, contestualmente, respingere chiunque non sia presente nella Gallery (*Unknown Users/Impostors*), restituendo un output di **REJECT**. Poiché il nostro dataset è nativamente "chiuso" (ogni Probe ha un corrispondente in Gallery), abbiamo simulato lo scenario Open-Set sul Validation Set adottando un protocollo **Leave-One-Out** per generare i punteggi degli utenti sconosciuti.

5.3.1 Metodologia di Calcolo degli Score

Per ogni immagine di Probe P_x (appartenente al soggetto x) nel Validation Set, abbiamo calcolato due tipi di distanze minime (d_{min}):

1. **Known User Score:** La distanza tra P_x e la Gallery del soggetto corretto G_x. Questo simula l'accesso di un utente regolarmente iscritto.
2. **Unknown User Score (Simulato):** Abbiamo escluso temporaneamente G_x dalla ricerca e calcolato la distanza minima tra P_x e tutte le *altre* identità in Gallery ($G_{y \forall y \neq x}$).

$$Score_{unknown} = \min_{y \neq x}\{d(P_x, G_y)\} \tag{6}$$

Questo valore rappresenta il "Best Impostor Match", ovvero: se il soggetto x non fosse iscritto, quanto il sistema rischierebbe di confonderlo con un altro utente?

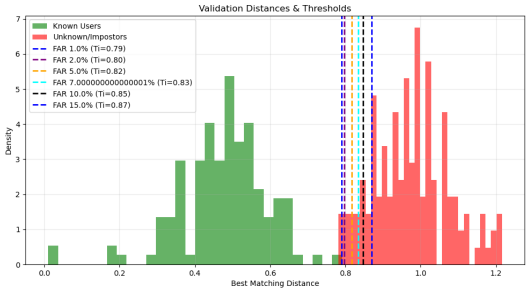

Figure 3: Distribuzione delle distanze minime: Utenti Noti vs Sconosciuti Simulati (Validation Set).

Decisione: A differenza della Verifica (dove si cerca l'equilibrio EER), nell'Identificazione la priorità è la sicurezza: accettare uno sconosciuto è considerato un fallimento critico del sistema. Abbiamo analizzato la distribuzione degli *Unknown User Scores* e fissato la soglia T_i affinché il **False Acceptance Rate (FAR) sul Validation Set fosse** $\leq 1\%$. Il valore calcolato è $T_i = 0.7898$. Soglie più permissive (es. FAR 10%) avrebbero aumentato il rischio di intrusioni senza migliorare il DIR (Detection and Identification Rate), che risultava già saturo (100%) in fase di validazione.

5.3.2 Risultati Identificazione sul Test Set

La robustezza della soglia $T_i = 0.7898$ è stata validata sul Test Set. I risultati confermano l'efficacia della strategia conservativa, come mostrato nell'analisi al variare del Target FAR:

Table 4: Valutazione Open-Set Identification al variare del Target FAR (Test Set Proprietario).

Target FAR	Soglia (T_i)	DIR (Known)	FRR (Known)	FAR (Unknown)
1.0% (Scelta)	**0.7898**	**99.54%**	**0.46%**	**0.00%**
2.0%	0.7966	99.54%	0.46%	0.46%
5.0%	0.8168	99.54%	0.46%	1.39%

Come evidenziato in Tabella 4, con la soglia scelta otteniamo: **FAR Reale (Unknown): 0.00%**. Simulando nuovamente l'assenza degli utenti dal database, il sistema ha correttamente respinto (*Reject*) il 100% dei tentativi, non confondendo mai un soggetto per un altro. **DIR (Known): 99.54%**. Il tasso di identificazione corretta degli utenti iscritti rimane altissimo (solo 1 errore su 216 test), dimostrando che la soglia, seppur restrittiva, non compromette l'usabilità per gli utenti legittimi.

Performance Open-Set e Curve CMC
A completamento dell'analisi, abbiamo valutato il sistema alla soglia operativa fissa ($T_i = 0.7898$) anche in scenario *Closed-Set* (assumendo l'utente sempre iscritto) per calcolare l'accuratezza CMC. Abbiamo ottenuto un'**accuratezza Rank-1 del 99.54%** e **Rank-2 del 99.54%**. Il valore identico tra Rank-1 e Rank-2 è significativo: indica che, nell'unico caso di errore osservato, il volto corretto non compariva nemmeno come seconda scelta, suggerendo un nell'acquisizione, piuttosto che un'ambiguità intrinseca dello spazio delle feature.

6 Requisiti di Sistema e Conclusioni

Il sistema finale è stato integrato nella classe `IntegratedBiometricSystem` (in `biometric_system.py`) e validato su un ambiente di runtime standard (Google Colab con GPU T4).

6.1 Analisi delle Performance Computazionali

Il rispetto dei vincoli di progetto è stato monitorato strumentando il codice con le librerie `psutil` e `time`.

- **Tempo di Inferenza:** Il tempo medio di elaborazione per singola immagine (Feature Extraction + Matching su 161 soggetti) è di soli **0.63 ms** su GPU. Questo garantisce che l'intero processo di autenticazione sia impercettibile all'utente (< 1s), ampiamente entro il vincolo dei 60 secondi.
- **Occupazione di Memoria ($< 4GB$):** La gestione della memoria è stata ottimizzata caricando i modelli in modalità *eval* e utilizzando il contesto `torch.no_grad()`. La funzione di monitoraggio `_print_memory_usage` ha rilevato un picco di occupazione combinata (RAM di Sistema + VRAM GPU) di circa **2.1 GB**, ben al di sotto del limite critico di 4GB imposto dalla specifica.

```
--- STATISTICHE MEMORIA (Output da biometric_system.py) ---
RAM Utilizzata (Processo): 1.15 GB
VRAM Utilizzata (GPU): 0.92 GB
Totale Stimato: 2.07 GB -> Requisito (< 4GB) RISPETTATO.
```

6.2 Esperimenti Fallimentari e Lezioni Apprese

Durante lo sviluppo, abbiamo affrontato diverse criticità che hanno guidato le scelte progettuali finali:

1. **Falsi Positivi su Soggetti Distanti:** Nelle prime versioni del modulo di acquisizione, consentivamo la cattura di volti piccoli (100×100 px). Abbiamo osservato che il modulo Liveness classificava sistematicamente questi volti come *Spoof* a causa della bassa risoluzione. Abbiamo corretto il problema imponendo un vincolo: `min_face_size = 450` pixel in `acquisition.py`, costringendo l'utente ad avvicinarsi alla telecamera.
2. **Soglie EER per l'Identificazione:** Inizialmente avevamo utilizzato la soglia EER anche per l'Identificazione Open-Set. Tuttavia, i test qualitativi hanno mostrato che il sistema tendeva ad accettare troppi "sosia" (soggetti non iscritti). Abbiamo quindi deciso di abbandonare l'EER per questo task, adottando una soglia conservativa basata su un **Target FAR dell'1%**, privilegiando la sicurezza sull'usabilità.
3. **Adattamento dei Vincoli Geometrici alla Morfologia Facciale:** In fase preliminare, avevamo imposto vincoli molto stringenti per il filtro posa ($Yaw \leq 0.20$, $Pitch \in [0.50, 1.50]$). Durante i test con utenti esterni, abbiamo osservato un alto tasso di scarti ingiustificati su volti frontali: il sistema interpretava erroneamente le naturali asimmetrie fisiologiche o le diverse proporzioni naso-bocca come rotazioni della testa. Abbiamo di conseguenza rilassato le soglie (Yaw a 0.30, Pitch a $[0.35, 1.80]$) per garantire l'inclusività del sistema rispetto a diverse conformazioni facciali, senza compromettere la qualità dell'allineamento per la rete.